Reproducing and Extending the MTTD Dynamic Model-Selection Framework: Five ML Models on a 2023 Cricket World Cup Workload with CloudSim Evaluation

Abhinav Mishra, Garv Gujral, and Shashank Yadav

Abstract

Nashaat *et al.* (2026) proposed MTTD, a Monitor–Train–Test–Deploy framework that trains several workload predictors, deploys the one with the lowest error score, and retrains when an adaptive threshold is breached on two consecutive cycles. They evaluated three models (ARIMA, SVR, Random Forest) on FIFA World Cup server logs. We reproduce the framework with five models, adding XGBoost and an LSTM, and apply it to an hourly workload derived from daily Wikipedia page views (user traffic) of the 2023 ICC Cricket World Cup. We also add an option that re-selects the active model every day or every hour, and replay every forecast in a CloudSim data-centre simulation. In the standard MTTD run, the test phase selected Random Forest (Score 6.49%), one retraining event re-selected it, and it remained active for all 1,080 deployment hours (Score 32.78%), although the best-scoring model varied from day to day. Re-selecting every day or every hour made the active model switch 27 and 77 times and lowered the score to 21.41% and 18.96%. In CloudSim 3.0.3, standard MTTD provisioned 7,475 VM-hours against 6,981 needed and violated the SLA in 145 of 1,080 hours; hourly reselection reduced wasted VM-hours from 1,077 to 702 but raised SLA-violating hours to 158, and daily reselection to 263.

Index Terms

Cloud autoscaling, workload prediction, dynamic model selection, concept drift, flash crowds, CloudSim, XGBoost, LSTM.

I. Introduction

ELASTIC cloud platforms promise capacity that tracks demand, but the controllers that deliver it are mostly reactive. A threshold autoscaler, such as the Kubernetes Horizontal Pod Autoscaler [34], adds replicas only after utilisation has already crossed a target, so the first minutes of a surge are served by an undersized fleet [11], [12], [31]. Production systems at hyperscale therefore combine reactive control with forecasting [32], [33].

Live sports events are a hard case for forecasting. Audience attention concentrates in match windows, swings between fixtures of different importance, and peaks during knockout games and finals. Such flash crowds are well documented for web workloads since the 1998 FIFA World Cup [13], [14]. A forecaster trained on the group stage sees a different distribution from the one it must serve during the semi-finals, an instance of concept drift [35].

Comparative studies repeatedly find that no single model dominates across workload patterns [3], [19], [25], [26]. Nashaat *et al.* [1] responded with MTTD, a Monitor–Train–Test–Deploy loop that trains several candidates, deploys the best, and retrains when an error-amplified threshold is breached on two consecutive cycles. Their evaluation used the 1998 and 2018 FIFA World Cup server logs and three candidates (ARIMA, SVR, Random Forest), and stopped at prediction accuracy.

This paper reproduces MTTD and extends it. Its contributions are:

1) **Reproduction of the MTTD pipeline.** We implement all four phases of [1], including its performance score and two-strike adaptive retraining threshold, and repeat its synthetic-workload experiments.
2) **Five candidate models.** We add gradient boosting (XGBoost [8]) and a recurrent network (LSTM [9]) to the original ARIMA, SVR and Random Forest candidates.

The authors are with Netaji Subhas University of Technology, New Delhi 110075, India.
Code, workload generator and simulation: https://github.com/garv-codes/ML-workload-prediction.

3) **A 2023 Cricket World Cup workload.** In place of the FIFA logs, we disaggregate 56 daily English-Wikipedia page-view counts (user traffic) for the tournament article into 1,344 hourly intervals.
4) **Periodic reselection.** We add an option that re-chooses the active model every N hours from its recent error, and run it with $N = 24$ (daily) and $N = 1$ (hourly).
5) **CloudSim evaluation.** Every run's hourly forecasts are replayed in a CloudSim [2] data-centre simulation that provisions VMs from the forecast and executes the real load.

Section II reviews related work. Section III presents the method. Section IV reports the experimental setup and results. Section V concludes.

II. Related Work

Workload prediction for autoscaling has been surveyed extensively [11], [12], [28], [29]. We group prior work into four families, summarised in Fig. 1 and Table I.

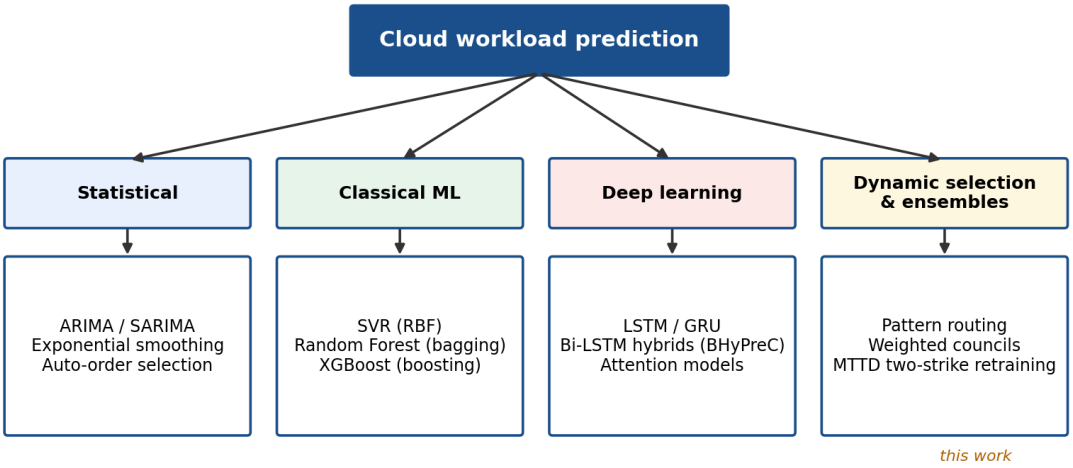


Fig. 1. Taxonomy of cloud workload prediction approaches. This work extends the dynamic-selection family.

A. Statistical Models

ARIMA models a series through autoregressive, differencing and moving-average terms [4]. Calheiros *et al.* [3] used ARIMA to forecast web requests and evaluated its effect on QoS in CloudSim. Automatic order selection [5] removes manual ACF/PACF tuning. Early predictive scalers such as PRESS [17] and CloudScale [18] paired signal-processing or Markov predictors with corrective control. ARIMA assumes linear, locally stationary dynamics, so strong intraday seasonality degrades it unless a seasonal term is modelled.

B. Classical Machine Learning

Support vector regression [6] and Random Forests [7] recast forecasting as supervised regression on a sliding window of lags. Empirical comparisons [16], [19] evaluated such learners against time-series models, and Kim *et al.* [19] found that no single predictor is best for all workload patterns. Later studies applied these learners directly to elastic provisioning [23], [30]. Gradient boosting with XGBoost [8] adds shrinkage and regularised tree growth.

C. Deep Learning

LSTMs [9] capture long-range temporal dependence through gated memory. Kumar and Singh [21], [22] trained neural forecasters with evolutionary and self-directed learning. Bi *et al.* [24] combined bidirectional and grid LSTMs (BG-LSTM). BHyPreC [10] stacks Bi-LSTM, LSTM and GRU layers for VM CPU prediction.

D. Dynamic Selection and Ensembles

Because no single model dominates, several works select or combine models online. Liu *et al.* [25] discriminate workload patterns and route each to a matching predictor. AME-WPC [26] combines several forecasting methods, and Messias *et al.* [27] combine time-series models with a genetic algorithm. CloudInsight [20] maintains a council of predictors and reweights them over time. MTTD [1] instead deploys one active model and retrains all candidates only when an error-amplified threshold is breached twice in a row.

TABLE I
REPRESENTATIVE WORKLOAD-PREDICTION APPROACHES

Ref.	Approach	Family	Dynamic selection	Evaluation scope
[3]	ARIMA	Statistical	No	Forecast accuracy and QoS impact in CloudSim
[15]	ARMA + look-ahead control	Statistical	No	Forecasting with autoscaling control
[16]	NN, linear regression	Classical ML	No	Forecast accuracy
[19]	21 predictors compared	Mixed	No	24 synthetic workloads; cost and deadline-miss simulation
[21]	NN + adaptive DE	Deep / evolutionary	No	Forecast accuracy
[10]	Bi-LSTM + LSTM + GRU (BHyPreC)	Deep learning	No	VM CPU forecast accuracy
[25]	Pattern-routed predictors	Dynamic	Yes (by pattern)	Forecast accuracy vs. ARIMA, SVM, LR
[20]	Predictor council (CloudInsight)	Ensemble	Yes (reweighting)	Real traces; trace-based provisioning simulation
[1]	MTTD: ARIMA, SVR, RF	Dynamic	Yes (two-strike)	Forecast accuracy on FIFA 1998 and 2018 logs
This work	MTTD: + XGBoost, LSTM	Dynamic	Yes (two-strike; periodic)	Forecast accuracy and CloudSim provisioning

This work follows [1] and adds two models, a new workload, periodic model reselection and a CloudSim provisioning evaluation (last row of Table I).

III. METHODOLOGY

A. Two-Stage Pipeline

The system separates prediction from provisioning (Fig. 2). Stage 1, in Python (`mttd_bridge.py`), runs the online MTTD loop over the hourly trace and writes, for every deployment hour, the observed load, the active model's one-step-ahead forecast, the active model's name and any retraining event to a CSV file. Stage 2, in Java (`ProvisioningSimulation.java`), reads that file and replays each hour in CloudSim.

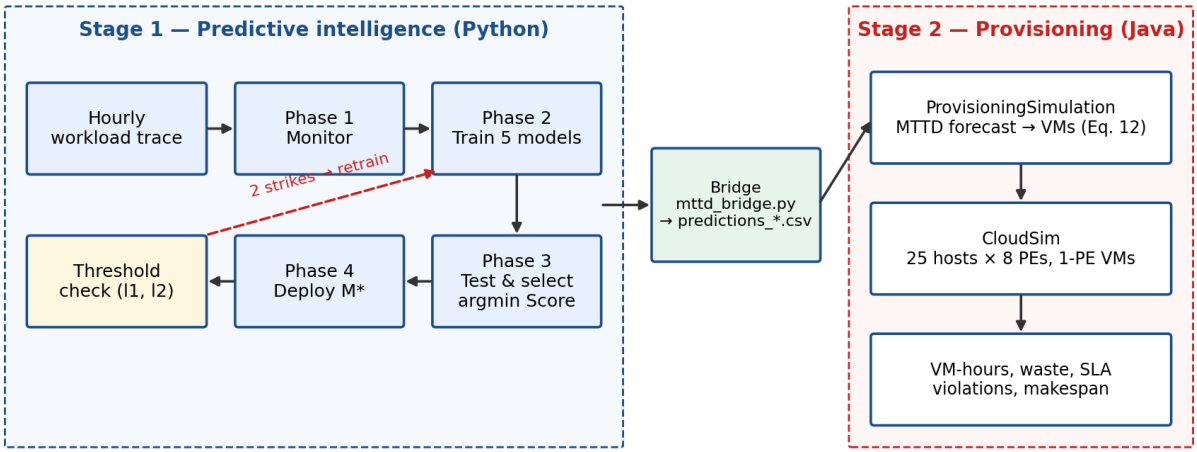


Fig. 2. Two-stage pipeline: the Python MTTD loop (Stage 1), the CSV file of hourly forecasts, and the Java CloudSim simulation (Stage 2).

B. Metrics

Let y_t be the number of requests in hour t and $\hat{y}_t^{(i)}$ the one-step-ahead forecast of candidate $i \in \mathcal{M} = \{\text{ARIMA}, \text{SVR}, \text{RF}, \text{XGB}, \text{LSTM}\}$, made from observations up to $t-1$. Over an evaluation window of N hours we use the error measures of [1], [36]:

$$\text{RMSE}_i = \sqrt{\frac{1}{N}\sum_{t=1}^{N}\left(y_t - \hat{y}_t^{(i)}\right)^2}, \tag{1}$$

$$\text{MAE}_i = \frac{1}{N}\sum_{t=1}^{N}\left|y_t - \hat{y}_t^{(i)}\right|. \tag{2}$$

MTTD ranks models by the normalised, weighted score of [1] (its Eq. 4) with $w_1 = w_2 = 0.5$:

$$\text{Score}_i = 100\left(w_1\frac{\text{RMSE}_i}{\sqrt{\frac{1}{N}\sum_t y_t^2}} + w_2\frac{\text{MAE}_i}{\frac{1}{N}\sum_t y_t}\right). \tag{3}$$

The active model is the candidate with the lowest score on the test window:

$$M^* = \arg\min_{i\in\mathcal{M}} \text{Score}_i. \tag{4}$$

On the synthetic benchmarks we also report $\text{MAPE}_i = \frac{100}{N}\sum_t |y_t - \hat{y}_t^{(i)}|/|y_t|$.

C. Two-Strike Retraining Threshold

The active model is deployed for cycles of 24 hours. Let S_t be its score on cycle t. Following [1], the threshold for the next cycle is set from the latest healthy score with weight l_1, and set with weight l_2 after a first breach:

$$\text{Threshold}_{t+1} = S_t + l_1 S_t, \qquad l_1 = 0.75, \tag{5}$$

$$\text{Threshold}_{t+2} = S_{t+1} + l_2 S_{t+1}, \qquad l_2 = 0.50. \tag{6}$$

A strike counter k implements the two-strike rule, retraining at t if and only if $k_t = 2$:

$$k_t = \begin{cases} 0, & S_t \le \text{Threshold}_t, \\ k_{t-1} + 1, & S_t > \text{Threshold}_t. \end{cases} \tag{7}$$

On retraining, all five candidates are refitted on the latest 240 hours, re-tested on the last 24 hours, and the lowest-scoring model becomes active. Algorithm 1 and Fig. 3 summarise the loop. All candidates forecast every hour so that their scores can be logged, but only the active model's forecasts are written to the output file.

Algorithm 1 Online MTTD with two-strike retraining

Require: series y, candidates $\mathcal{M}$, T_{tr}=240 h, T_{te}=24 h, cycle n=24 h, window w=24, l_1=0.75, l_2=0.50

$t \leftarrow T_{tr} + T_{te}$ ▷ Phase 1: monitor
fit every $m \in \mathcal{M}$ on $y[t{-}T_{tr}{-}T_{te} : t{-}T_{te}]$ ▷ Phase 2
$S_m \leftarrow$ Score of rolling forecast of $y[t{-}T_{te} : t]$ for all m
$M^* \leftarrow \arg\min_m S_m$; $\theta \leftarrow S_{M^*}(1{+}l_1)$; $k \leftarrow 0$ ▷ Phase 3
while $t < |y|$ **do** ▷ Phase 4: deploy
  forecast $y[t : t{+}n]$ one step ahead with M^*, observing truth
  $S \leftarrow$ Score of this cycle
  if $S \leq \theta$ **then**
    $k \leftarrow 0$; $\theta \leftarrow S(1{+}l_1)$
  else
    $k \leftarrow k + 1$
    if $k = 1$ **then**
      $\theta \leftarrow S(1{+}l_2)$
    else
      $k \leftarrow 0$; $t' \leftarrow t + n$
      refit all m on $y[t'{-}T_{tr}{-}T_{te} : t'{-}T_{te}]$
      re-test on $y[t'{-}T_{te} : t']$; $M^* \leftarrow \arg\min$; $\theta \leftarrow S_{M^*}(1{+}l_1)$
  $t \leftarrow t + n$

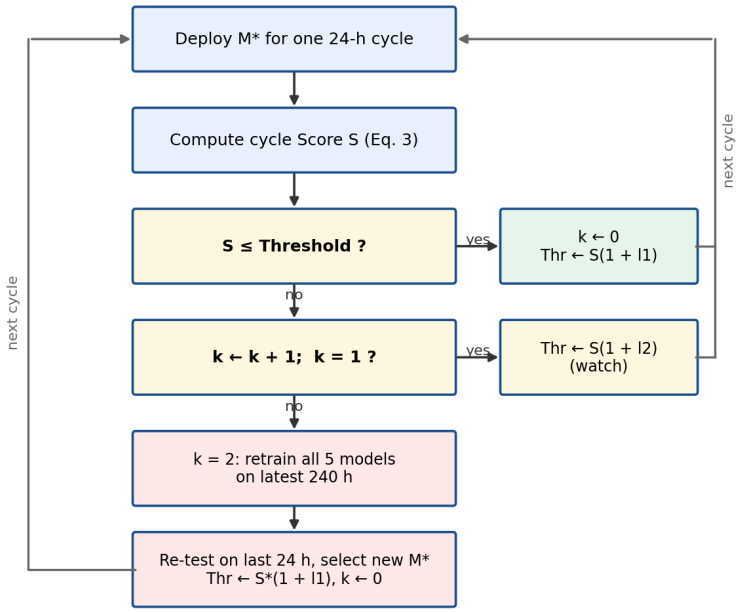


Fig. 3. Retraining and model-reselection flowchart implementing Eqs. (5)–(7).

D. Data Engineering: Daily-to-Hourly Disaggregation

Daily page views D_d for the English Wikipedia article "2023 Cricket World Cup" were exported from the Wikimedia Pageviews tool [39] for 1 Oct–25 Nov 2023 (56 days) with agent = user, which excludes bots and automated traffic. Let $h \in \{0, \ldots, 23\}$ be the hour of day. The match window $\mathcal{P} = \{14, \ldots, 21\}$ receives 70% of the day's traffic and the 16 other hours share 30%:

$$b_{d,h} = \begin{cases} 0.70\, D_d/8, & h \in \mathcal{P}, \\ 0.30\, D_d/16, & h \notin \mathcal{P}. \end{cases} \tag{8}$$

Each hour is perturbed by bounded uniform noise:

$$\tilde{y}_{d,h} = b_{d,h}\, \varepsilon_{d,h}, \qquad \varepsilon_{d,h} \sim \mathcal{U}(0.92,\, 1.08). \tag{9}$$

The noisy hours are renormalised so that each day keeps its real total, and rounded with the residual assigned to the largest hour:

$$y_{d,h} = \mathrm{round}\left(\tilde{y}_{d,h} \, \frac{D_d}{\sum_{h'} \tilde{y}_{d,h'}} \right), \qquad \sum_h y_{d,h} = D_d. \tag{10}$$

A fixed random seed (42) makes the trace reproducible.

E. Periodic Reselection (Extension)

In standard MTTD the active model changes only at a retraining event. As an extension, `mttd_bridge.py` accepts an option `-reselect-every` N: every N hours the active model is replaced by the candidate with the lowest score over the most recent $W = 24$ hours of logged forecasts,

$$M_t^* = \arg\min_{i \in \mathcal{M}} \mathrm{Score}_i\big(y_{t-W:t},\, \hat{y}^{(i)}_{t-W:t}\big), \qquad t \equiv t_0 \pmod N. \tag{11}$$

The two-strike retraining rule stays active and is evaluated on the forecasts actually used. We run $N = 24$ (daily) and $N = 1$ (hourly).

F. CloudSim Provisioning Model

For each hour, the number of VMs is set from the forecast with headroom $\eta = 1.10$ and a cap of $V_{\max} = 150$:

$$V_t = \min\big(V_{\max}, \max(1, \lceil \eta \, \hat{y}_t / C \rceil)\big), \tag{12}$$

where $C = 4{,}000$ is the number of requests one VM serves per hour (900 MI per request on a 1,000-MIPS VM). The VMs needed if the load were known exactly are

$$V_t^{\mathrm{req}} = \min\big(V_{\max}, \max(1, \lceil y_t / C \rceil)\big). \tag{13}$$

The hour's real requests are executed in CloudSim as cloudlets of 200 requests (180,000 MI, 180 s on one VM), assigned round-robin to time-shared VMs. An hour violates the SLA if the last cloudlet finishes after one hour:

$$\mathrm{SLA}_t = \mathbb{1}\big[T_t > 3600\,\mathrm{s}\big]. \tag{14}$$

We report VM-hours provisioned and needed, wasted VM-hours ($\sum_t \max(0, V_t - V_t^{\mathrm{req}})$), under-provisioned hours ($V_t < V_t^{\mathrm{req}}$), SLA-violating hours and the average completion time.

IV. Experimental Setup and Results

A. Setup

Table II lists the CloudSim parameters and Table III the candidate models. All window-based models use the last 24 hourly values as features; the training window is 240 hours, the test window 24 hours and the deployment cycle 24 hours, matching the hourly configuration of [1]. Grid searches use 3-fold `GridSearchCV` with negative MSE [37]; the LSTM uses PyTorch [38].

TABLE II
CloudSim Simulation Parameters (`ProvisioningSimulation.java`, CloudSim 3.0.3)

Parameter	Value
Data centre	25 hosts × 8 PEs × 1,000 MIPS, 16 GB RAM, time-shared VM scheduler
VM	1 PE, 1,000 MIPS, 512 MB RAM, time-shared cloudlet scheduler
Capacity C	4,000 requests per VM per hour (900 MI per request)
Cloudlet	200 requests = 180,000 MI (180 s on one VM)
Headroom η / cap $V_{\max}$	1.10 / 150 VMs
Time step and SLA	3,600 s; all cloudlets of the hour must finish within 3,600 s
Horizon	1,080 hours (hours 264–1343)
Software	CloudSim 3.0.3, JDK 26

TABLE III
CANDIDATE PREDICTORS ON THE CRICKET TRACE (`MTTD_BRIDGE.PY`)

Model	Configuration and search grid
ARIMA	`pmdarima.auto_arima` [5], non-seasonal, $p, q \leq 5$, d by unit-root test, stepwise; updated with each new observation
SVR	RBF; $C \in \{1, 10, 100\}$, $\epsilon \in \{0.001, 0.01\}$, $\gamma \in \{0.01, 0.1\}$; inputs scaled by the training maximum
Random Forest	100 trees; max depth $\in \{10, 40\}$; min samples/leaf $\in \{1, 3\}$
XGBoost	trees $\in \{100, 200\}$; max depth $\in \{3, 6\}$; learning rate $\in \{0.05, 0.1\}$
LSTM	1 layer $\times$ 32 units + linear head; Adam, lr 0.01; 60 epochs; batch 32; z-score inputs; seed 42

B. Dataset Characteristics

Table IV summarises the trace and Fig. 4 plots it. Within each day the trace is a two-level step with noise; across days it follows the fixture schedule, rising through the semi-finals (15–16 Nov) to the Final (19 Nov) and collapsing after the tournament (20–25 Nov, hours 1200–1343).

TABLE IV
THE 2023 CRICKET WORLD CUP HOURLY TRACE

Property	Value
Source	Wikimedia Pageviews, en.wikipedia, "2023 Cricket World Cup", agent = user [39]
Span	1 Oct – 25 Nov 2023; 56 days; 1,344 hourly intervals
Total workload	29,957,762 requests (equal to the sum of the daily counts)
Deployment window	Hours 264–1343: 25,636,322 requests; mean 23,737 per hour
Peak	117,612 requests at 17:00, 19 Nov 2023 (the Final; hour 1193)
Hourly profile	8.75% of daily traffic per match hour (14:00–21:00), 1.875% per other hour; noise $\mathcal{U}(0.92, 1.08)$

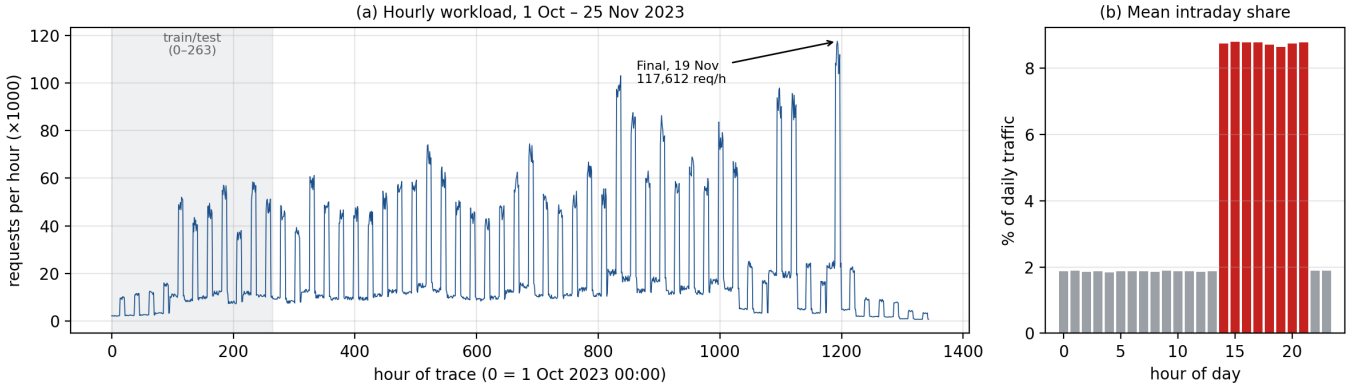


Fig. 4. (a) Hourly cricket workload, 1 Oct – 25 Nov 2023; the shaded region is the initial training and test window. (b) Mean share of daily traffic by hour of day.

C. Synthetic Benchmark

We ran each predictor on the four synthetic patterns of [1] (697 minutes, 70/30 split, 100-point test window, window of 5) using `run_benchmark.py` (Table V, Fig. 5). ARIMA scored best on all four patterns. The original paper reports SVR as the best model on its random workload [1]; in our reproduction SVR was worst on three of the four patterns.

TABLE V
SYNTHETIC BENCHMARKS: SCORE % (MAPE IN PARENTHESES); BEST PER PATTERN IN BOLD

Model	Uniform (bell)	Linear low (0.35)	Linear high (2.31)	Bounded random walk
ARIMA	**1.51 (1.42%)**	**0.49 (0.44%)**	**0.08 (0.07%)**	**3.63 (3.28%)**
SVR	11.11 (11.49%)	50.95 (49.73%)	54.28 (54.05%)	7.73 (7.00%)
Random Forest	3.95 (3.96%)	10.56 (9.61%)	10.25 (9.29%)	3.96 (3.66%)
XGBoost	4.35 (4.31%)	10.46 (9.51%)	10.28 (9.32%)	3.83 (3.55%)
LSTM	3.70 (3.80%)	2.29 (2.17%)	2.14 (2.06%)	3.69 (3.40%)

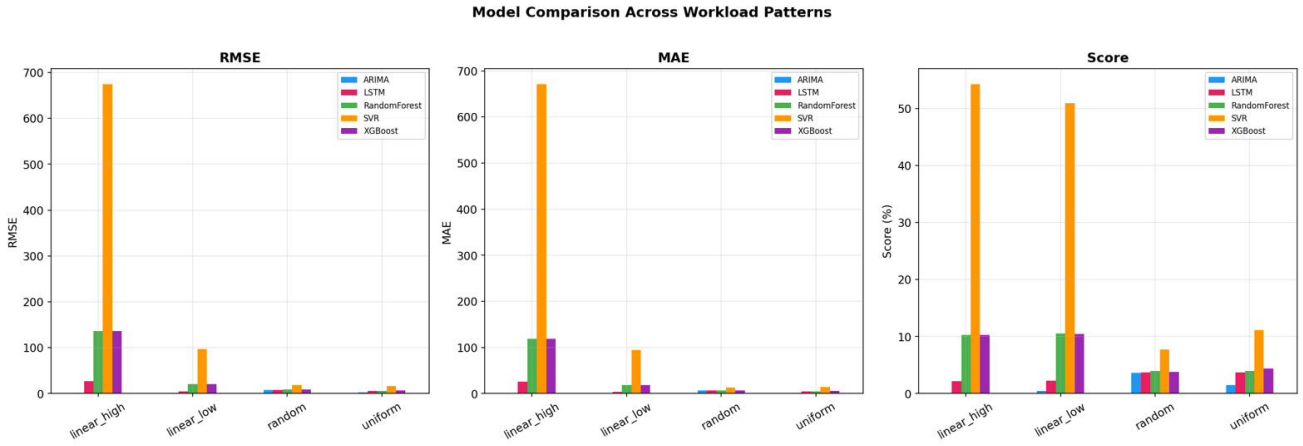


Fig. 5. RMSE, MAE and Score of the five predictors across the four synthetic workload patterns.

D. Standard MTTD on the Cricket Trace

The initial test window (hours 240–263) ranked Random Forest first (Table VI). One retraining event occurred, at hour 552, and the re-test again selected Random Forest, which therefore remained active for all 1,080 deployment hours.

TABLE VI
TEST-PHASE SCORES (%) USED FOR MODEL SELECTION

Test window	RF	XGBoost	LSTM	SVR	ARIMA	Selected
Initial (hours 240–263)	**6.49**	7.07	7.55	11.75	29.18	Random Forest
Retrain at hour 552	**5.42**	7.15	9.44	7.48	33.26	Random Forest

Table VII lists every threshold breach of the active model. Ten breaches produced one retraining event; eight single-cycle breaches were absorbed because the following cycle stayed below the threshold set by Eq. (6). From cycle 33 onwards the scores were high (23% to 523%), but each breach raised the next threshold to 1.5 times a large score, so no second consecutive breach occurred.

TABLE VII
THRESHOLD BREACHES OF THE ACTIVE MODEL ACROSS THE 45 DEPLOYMENT CYCLES

Cycle	Start hour	Score%	Threshold%	k	Outcome (next cycle)
2	288	10.69	10.51	1	Absorbed ($7.51 \leq 16.04$)
11	504	20.75	6.94	1	Watch
12	528	97.48	31.13	2	Retrain $\rightarrow$ Random Forest
24	816	29.87	11.07	1	Absorbed ($17.21 \leq 44.81$)
27	888	13.65	8.62	1	Absorbed ($4.98 \leq 20.48$)
31	984	10.89	8.57	1	Absorbed ($5.26 \leq 16.33$)
33	1032	90.14	9.20	1	Absorbed ($102.85 \leq 135.22$)
37	1128	90.16	40.53	1	Absorbed ($80.70 \leq 135.24$)
40	1200	166.82	138.45	1	Absorbed ($130.00 \leq 250.23$)
44	1296	386.87	296.31	1	Absorbed ($523.03 \leq 580.30$)

Fig. 6 shows the daily score of every candidate. The best-scoring model changed frequently (Table VIII): Random Forest had the lowest score in 14 cycles, SVR and XGBoost in 10 each, ARIMA in 8 and the LSTM in 3. Over the whole deployment, SVR had the lowest score (22.41%) and Random Forest, the active model, the highest (32.78%).

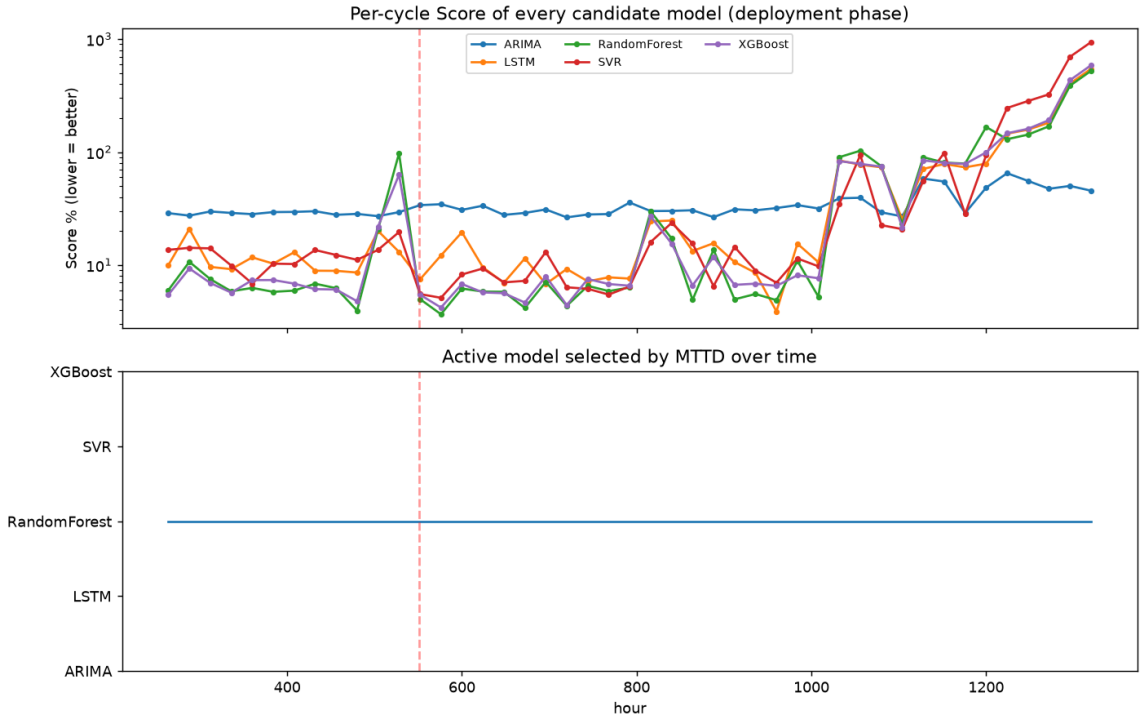


Fig. 6. Standard MTTD. Top: daily score of all five candidates (log scale). Bottom: active model; the dashed line marks the retraining event at hour 552.

TABLE VIII
PER-CANDIDATE ACCURACY IN THE STANDARD RUN (1,080 HOURS)

Candidate	Overall Score%	Cycles with lowest score	Hours active
SVR	22.41	10	0
LSTM	29.72	3	0
ARIMA	30.61	8	0
XGBoost	30.66	10	0
Random Forest	32.78	14	1,080

E. Periodic Reselection

Table IX compares the standard run with daily and hourly reselection. Figs. 7 and 8 show the per-cycle scores and the active model over time, and Fig. 9 compares the forecasts with the actual load.

TABLE IX
FORECAST ACCURACY OF THE THREE MTTD RUNS OVER 1,080 HOURS

Run	Score%	RMSE	MAE	Switches	Retrains	Hours active per model
Standard MTTD	32.78	13,649.96	5,945.88	0	1 (hour 552)	RF 1,080
Daily reselection	21.41	8,412.83	4,239.58	27	0	XGB 312, SVR 312, RF 168, LSTM 144, ARIMA 144
Hourly reselection	18.96	7,757.50	3,536.08	77	1 (hour 864)	XGB 320, SVR 289, RF 235, LSTM 118, ARIMA 118

Daily reselection lowered the score by 34.7% and hourly reselection by 42.2% relative to standard MTTD, and both ended below the best single candidate of the standard run (SVR, 22.41%). All five candidates were used in both runs. In the daily run the two-strike rule was never triggered, so no retraining took place; in the hourly run it triggered once, at hour 864, and the re-test selected Random Forest (6.78%).

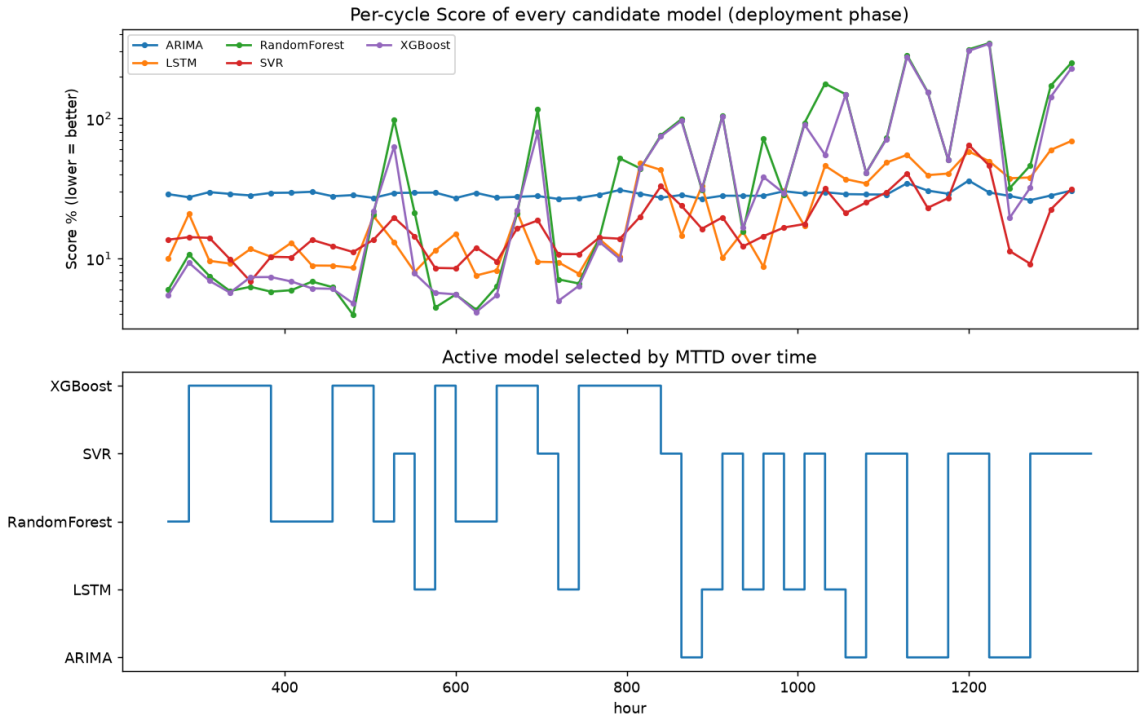


Fig. 7. Daily reselection. Top: daily score of all five candidates. Bottom: active model over time (27 switches).

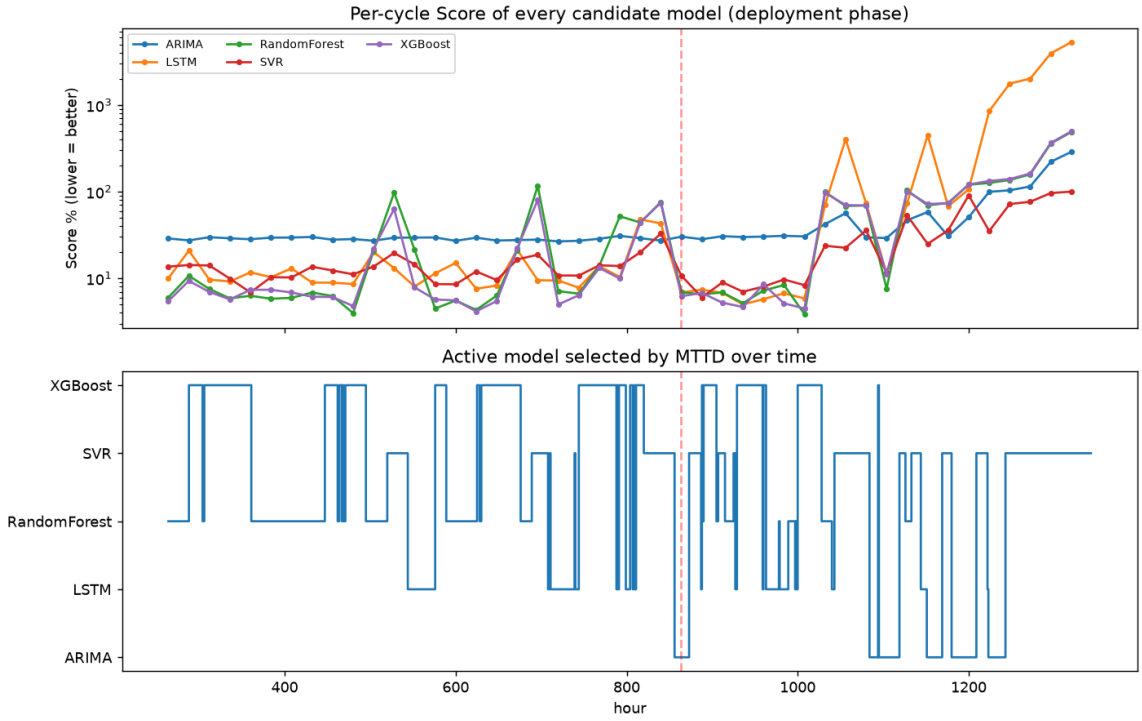


Fig. 8. Hourly reselection. Top: daily score of all five candidates. Bottom: active model over time (77 switches); the dashed line marks the retraining event at hour 864.

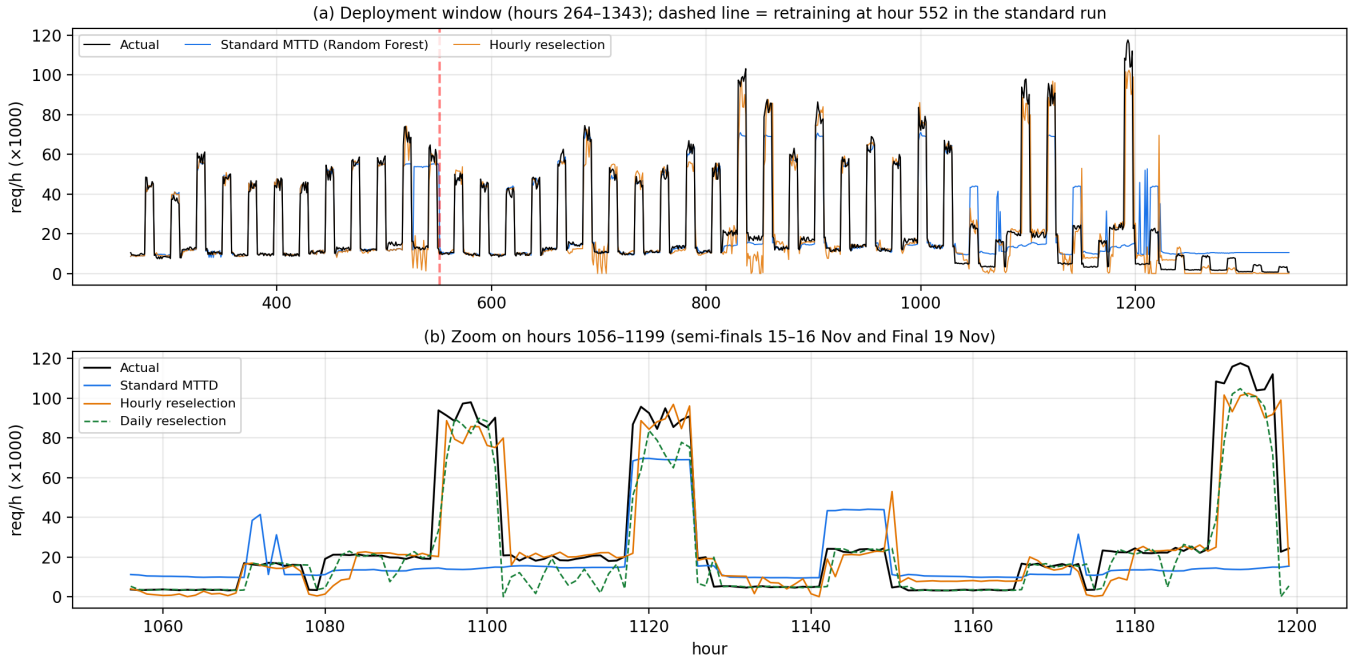


Fig. 9. (a) Actual load and the forecasts of standard MTTD and hourly reselection over the deployment window. (b) Zoom on the semi-finals and the Final, including daily reselection.

F. CloudSim Provisioning

Each run's forecasts were replayed in CloudSim over the 1,080 deployment hours. Fig. 10 shows the console output of the three simulations and Table X and Fig. 11 summarise them. The same load was used in all three, so the VMs needed for the exact load total 6,981 VM-hours in each case.

(a) Standard MTTD (predictions_all5.csv)

```
=== CloudSim provisioning driven by the MTTD forecasts ===
Time steps simulated            : 1080
VM-hours provisioned            : 7475
VM-hours needed (exact load)    : 6981
Wasted VM-hours (over-provision): 1077
Under-provisioned time steps    : 145 (13.4%)
SLA-violating time steps        : 145 (13.4%)
Average completion time (s)     : 3036

Per-step details written to cloudsim_C.csv
```

(b) Daily reselection (predictions_daily.csv)

```
=== CloudSim provisioning driven by the MTTD forecasts ===
Time steps simulated            : 1080
VM-hours provisioned            : 6802
VM-hours needed (exact load)    : 6981
Wasted VM-hours (over-provision): 460
Under-provisioned time steps    : 263 (24.4%)
SLA-violating time steps        : 263 (24.4%)
Average completion time (s)     : 3501

Per-step details written to cloudsim_E.csv
```

(c) Hourly reselection (predictions_hourly.csv)

```
=== CloudSim provisioning driven by the MTTD forecasts ===
Time steps simulated            : 1080
VM-hours provisioned            : 7312
VM-hours needed (exact load)    : 6981
Wasted VM-hours (over-provision): 702
Under-provisioned time steps    : 158 (14.6%)
SLA-violating time steps        : 158 (14.6%)
Average completion time (s)     : 3324

Per-step details written to cloudsim_D.csv
```

Fig. 10. CloudSim 3.0.3 console output for (a) standard MTTD, (b) daily reselection and (c) hourly reselection.

TABLE X
CLOUDSIM PROVISIONING RESULTS OVER 1,080 HOURS

Run	VM-hours	Wasted VMh	Under-prov. hours	SLA-violating hours	Avg completion (s)
Standard MTTD	7,475	1,077	145	145 (13.4%)	3,036
Daily reselection	6,802	460	263	263 (24.4%)	3,501
Hourly reselection	7,312	702	158	158 (14.6%)	3,324
Exact load (needed)	6,981	–	–	–	–

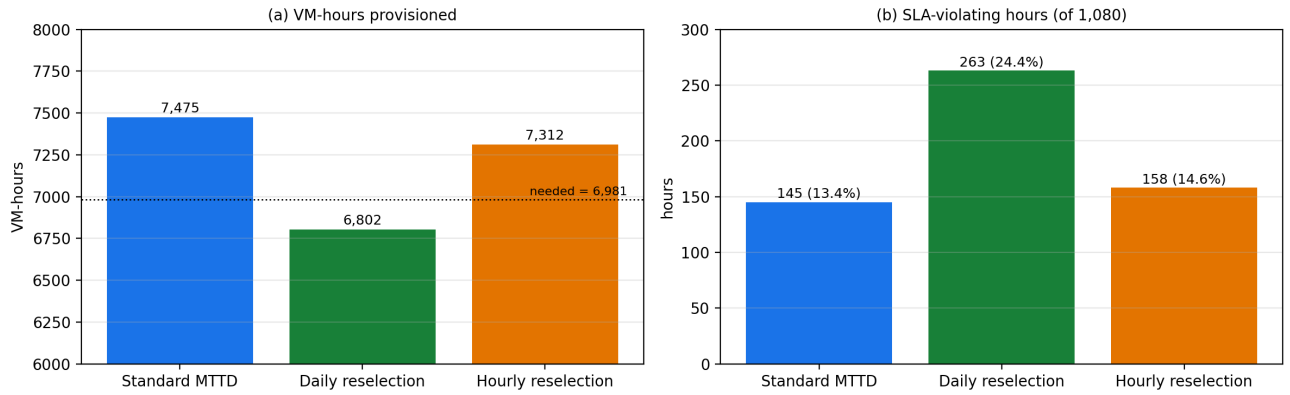


Fig. 11. (a) VM-hours provisioned by each run against the 6,981 VM-hours needed. (b) SLA-violating hours.

Standard MTTD provisioned 7.1% more VM-hours than needed and violated the SLA in 145 hours. Hourly reselection provisioned 4.7% more than needed, reduced wasted VM-hours by 34.8% (702 vs. 1,077), and violated the SLA in 158 hours. Daily reselection provisioned 2.6% fewer VM-hours than needed, reduced wasted VM-hours by 57.3% (460), and violated the SLA in 263 hours. In every run each SLA-violating hour was an under-provisioned hour. The lower forecast error of the reselection runs therefore came with less over-provisioning but more hours in which the forecast was too low.

G. Comparison with the Original Study

On the synthetic patterns our reproduction agrees with [1] that ARIMA performs well on the smooth uniform workload (the original reports a test-phase Score of 0.47%, ours is 1.51%), but differs on the random workload, where the original reports SVR as best. On the cricket trace, the pipeline behaved as designed: the test phase selected the lowest-scoring model, isolated bad cycles were absorbed, and two consecutive breaches triggered retraining and reselection. Because the active model changes only at retraining, Random Forest remained active even though the best-scoring model changed between cycles. Re-selecting periodically made both added models, XGBoost and the LSTM, active for part of the deployment; XGBoost was active for the most hours in the hourly run and, together with SVR, in the daily run.

H. Threats to Validity

Semi-synthetic data. Only the daily totals are measured; the hourly shape, match window and noise are modelled. *Proxy traffic.* Page views are not server requests and their burst statistics may differ from HTTP logs such as FIFA 1998 [13]. *Single trace and seed.* Results come from one tournament, one noise seed and one train/test configuration. *Cross-validation.* `GridSearchCV` with ordinary k-fold validation does not respect time order. *Non-seasonal ARIMA.* ARIMA was run without a seasonal term. *Simulation model.* VM capacity is fixed at 4,000 requests per hour; the simulation ignores VM start-up time and uses hour-long decision intervals.

V. Conclusion and Future Work

We reproduced the MTTD framework of Nashaat *et al.* with five candidate models instead of three and a 2023 Cricket World Cup workload instead of FIFA logs, and replayed its forecasts in CloudSim. In standard MTTD, Random Forest was selected at the test phase and again at the single retraining event, and remained active for all 1,080 hours with a score of 32.78%, while the best-scoring model changed from cycle to cycle. Re-selecting the model every day or every hour made MTTD switch between all five candidates and lowered the score to 21.41% and 18.96%. In CloudSim, standard MTTD violated the SLA in 145 of 1,080 hours; hourly reselection reduced wasted VM-hours by 34.8% but violated the SLA in 158 hours, and daily reselection in 263.

Future work includes validating on measured hourly or minute-level server logs of live sports events, reducing under-provisioning during sudden peaks, integrating MTTD forecasts with the Kubernetes Horizontal Pod Autoscaler [34], and using multivariate inputs such as CPU, memory and network metrics.

REFERENCES

[1] M. Nashaat *et al.*, "A dynamic machine learning approach for workload prediction in cloud environments," *Sci. Rep.*, vol. 16, art. no. 10983, 2026, doi: 10.1038/s41598-026-40777-z.

[2] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, "CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," *Softw. Pract. Exper.*, vol. 41, no. 1, pp. 23–50, 2011.

[3] R. N. Calheiros, E. Masoumi, R. Ranjan, and R. Buyya, "Workload prediction using ARIMA model and its impact on cloud applications' QoS," *IEEE Trans. Cloud Comput.*, vol. 3, no. 4, pp. 449–458, 2015.

[4] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.

[5] R. J. Hyndman and Y. Khandakar, "Automatic time series forecasting: The forecast package for R," *J. Stat. Softw.*, vol. 27, no. 3, pp. 1–22, 2008.

[6] H. Drucker, C. J. C. Burges, L. Kaufman, A. Smola, and V. Vapnik, "Support vector regression machines," in *Proc. NIPS*, 1997, pp. 155–161.

[7] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[10] M. E. Karim, M. M. S. Maswood, S. Das, and A. G. Alharbi, "BHyPreC: A novel Bi-LSTM based hybrid recurrent neural network model to predict the CPU workload of cloud virtual machine," *IEEE Access*, vol. 9, pp. 131476–131495, 2021.

[11] T. Lorido-Botran, J. Miguel-Alonso, and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments," *J. Grid Comput.*, vol. 12, no. 4, pp. 559–592, 2014.

[12] C. Qu, R. N. Calheiros, and R. Buyya, "Auto-scaling web applications in clouds: A taxonomy and survey," *ACM Comput. Surv.*, vol. 51, no. 4, art. no. 73, 2018.

[13] M. Arlitt and T. Jin, "A workload characterization study of the 1998 World Cup Web site," *IEEE Netw.*, vol. 14, no. 3, pp. 30–37, 2000.

[14] J. Jung, B. Krishnamurthy, and M. Rabinovich, "Flash crowds and denial of service attacks: Characterization and implications for CDNs and web sites," in *Proc. 11th Int. World Wide Web Conf.*, 2002, pp. 293–304.

[15] N. Roy, A. Dubey, and A. Gokhale, "Efficient autoscaling in the cloud using predictive models for workload forecasting," in *Proc. IEEE 4th Int. Conf. Cloud Comput.*, 2011, pp. 500–507.

[16] S. Islam, J. Keung, K. Lee, and A. Liu, "Empirical prediction models for adaptive resource provisioning in the cloud," *Future Gener. Comput. Syst.*, vol. 28, no. 1, pp. 155–162, 2012.

[17] Z. Gong, X. Gu, and J. Wilkes, "PRESS: PRedictive Elastic reSource Scaling for cloud systems," in *Proc. Int. Conf. Netw. Serv. Manage. (CNSM)*, 2010, pp. 9–16.

[18] Z. Shen, S. Subbiah, X. Gu, and J. Wilkes, "CloudScale: Elastic resource scaling for multi-tenant cloud systems," in *Proc. 2nd ACM Symp. Cloud Comput.*, 2011, art. no. 5.

[19] I. K. Kim, W. Wang, Y. Qi, and M. Humphrey, "Empirical evaluation of workload forecasting techniques for predictive cloud resource scaling," in *Proc. IEEE 9th Int. Conf. Cloud Comput.*, 2016, pp. 1–10.

[20] I. K. Kim, W. Wang, Y. Qi, and M. Humphrey, "Forecasting cloud application workloads with CloudInsight for predictive resource management," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1848–1863, 2022.

[21] J. Kumar and A. K. Singh, "Workload prediction in cloud using artificial neural network and adaptive differential evolution," *Future Gener. Comput. Syst.*, vol. 81, pp. 41–52, 2018.

[22] J. Kumar, A. K. Singh, and R. Buyya, "Self directed learning based workload forecasting model for cloud resource management," *Inf. Sci.*, vol. 543, pp. 345–366, 2021.

[23] J. Gao, H. Wang, and H. Shen, "Machine learning based workload prediction in cloud computing," in *Proc. 29th Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2020, pp. 1–9.

[24] J. Bi, S. Li, H. Yuan, and M. Zhou, "Integrated deep learning method for workload and resource prediction in cloud systems," *Neurocomputing*, vol. 424, pp. 35–48, 2021.

[25] C. Liu, C. Liu, Y. Shang, S. Chen, B. Cheng, and J. Chen, "An adaptive prediction approach based on workload pattern discrimination in the cloud," *J. Netw. Comput. Appl.*, vol. 80, pp. 35–44, 2017.

[26] K. Cetinski and M. B. Juric, "AME-WPC: Advanced model for efficient workload prediction in the cloud," *J. Netw. Comput. Appl.*, vol. 55, pp. 191–201, 2015.

[27] V. R. Messias, J. C. Estrella, R. Ehlers, M. J. Santana, R. C. Santana, and S. Reiff-Marganiec, "Combining time series prediction models using genetic algorithm to autoscale Web applications hosted in the cloud infrastructure," *Neural Comput. Appl.*, vol. 27, no. 8, pp. 2383–2406, 2016.

[28] M. Amiri and L. Mohammad-Khanli, "Survey on prediction models of applications for resources provisioning in cloud," *J. Netw. Comput. Appl.*, vol. 82, pp. 93–113, 2017.

[29] M. Masdari and A. Khoshnevis, "A survey and classification of the workload forecasting methods in cloud computing," *Cluster Comput.*, vol. 23, no. 4, pp. 2399–2424, 2020.

[30] R. Moreno-Vozmediano, R. S. Montero, E. Huedo, and I. M. Llorente, "Efficient resource provisioning for elastic cloud services based on machine learning techniques," *J. Cloud Comput.*, vol. 8, art. no. 5, 2019.

[31] N. R. Herbst, S. Kounev, and R. Reussner, "Elasticity in cloud computing: What it is, and what it is not," in *Proc. 10th Int. Conf. Autonomic Comput. (ICAC)*, 2013, pp. 23–27.

[32] K. Rzadca *et al.*, "Autopilot: Workload autoscaling at Google," in *Proc. 15th Eur. Conf. Comput. Syst. (EuroSys)*, 2020, art. no. 16.

[33] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in *Proc. 10th Eur. Conf. Comput. Syst. (EuroSys)*, 2015, art. no. 18.
[34] The Kubernetes Authors, “Horizontal Pod Autoscaling,” Kubernetes Documentation. [Online]. Available: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/
[35] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Comput. Surv.*, vol. 46, no. 4, art. no. 44, 2014.
[36] R. J. Hyndman and A. B. Koehler, “Another look at measures of forecast accuracy,” *Int. J. Forecast.*, vol. 22, no. 4, pp. 679–688, 2006.
[37] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
[38] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 8024–8035.
[39] Wikimedia Foundation, “Wikimedia REST API: Pageviews per-article endpoint.” [Online]. Available: https://wikimedia.org/api/rest_v1/